Adversarial Jailbreak Prompt Detection for Large Language Models Using a Hybrid Cache-Classifier Pipeline

1st Mrunal Mohan Vibhute
Dept. of Engineering Education (Artificial Intelligence Systems)
University of Florida
Gainesville, United States
mvibhute@ufl.edu

***Abstract*—Large language models remain vulnerable to jailbreak and prompt-injection attacks that can bypass alignment policies and expose downstream systems to harmful behavior. This project presents a hybrid defense pipeline that screens prompts before they reach a target LLM. The deployed system combines a seeded semantic cache for fast recognition of known attack patterns with a fine-tuned DistilBERT classifier for prompts that miss the cache. In parallel, an adversarial training loop mutates seed jailbreak prompts to generate harder examples and stress-test the detector.**

The final evaluation used a processed dataset of 24,037 prompts, an 80/20 train-test split, and an extra mutated stress-test set. Results showed that the calibrated baseline model worked best for deployment. In balanced mode, it achieved an F1-score of 0.9488 on the clean test set at a threshold of 0.51. A stricter mode with a threshold of 0.85 reduced false positives on more difficult wrapped prompts. Overall, the project contributes a practical two-stage detector, a user-facing Flask interface, adjustable operating modes, and an analysis of responsible-AI concerns such as false positives. The result is a lightweight, defensible prompt- screening workflow that emphasizes both model performance and deployment realism.

***Index Terms*—Artificial Intelligence, Jailbreak detection, DistilBERT, FAISS, Adversarial prompting**

I. Introduction

The problem of adversarial prompting is, at its core, a trust problem. When a user submits text to a large language model, the system has no reliable way to know whether that input is a genuine request or a carefully crafted attempt to subvert the model's behavior. Jailbreak prompts exploit this ambiguity, designed to confuse, flatter, or socially engineer a model into crossing the boundaries its developers intended [1], [2]. Prompt injection attacks take this threat further, embedding hidden instructions inside data that an LLM-powered application might ingest during normal operation [3]. The scope of the problem has grown considerably: a 2025 systematic review covering over a hundred studies found that prompt injection techniques have evolved from simple direct injections to sophisticated multimodal attacks, with success rates exceeding 90% against unprotected systems [17]. As these systems get deployed in higher-stakes settings, the gap between what a model is supposed to do and what it can be tricked into doing becomes a practical liability, not just a theoretical concern.

To understand why this problem is so persistent, it helps to think about how these models are trained. Large language models learn to be helpful by optimizing for responses that users rate positively. This training process makes them very good at following instructions, but it also creates an opening for adversarial manipulation: if an attacker can frame a harmful instruction in a way that looks like a normal request, the model's tendency to comply works against its own safety guidelines. The challenge is not simply that models can be tricked, but that the same properties that make them useful also make them exploitable [2]. This inherent tension means that purely capability-based defenses are unlikely to hold over time, which is what motivates the design of external, input-layer screening systems.

Major AI developers have recognized this and built defensive layers directly into their products. Anthropic's Constitutional AI framework trains Claude to refuse harmful instructions at the model level [7]. OpenAI applies reinforcement learning from human feedback to align GPT models against misuse, and maintains a moderation API specifically for flagging unsafe inputs [8]. Google's Gemini incorporates safety classifiers and policy filters as part of its inference pipeline, and has further operationalized this through ShieldGemma, a dedicated content moderation classifier built on the Gemma architecture and trained against the MLCommons safety taxonomy [9], [18]. Meta's Llama Guard family frames the problem as an input-output classification task, deploying a fine-tuned model as an external safeguard that operates alongside the target LLM [14]. These are sophisticated, well-resourced efforts, but they operate largely inside the model or require large fine-tuned guard models running in parallel. The question this project asks is a different one: what can be done at the input layer, before the model is ever queried, by a lightweight screening system that any developer could deploy in front of their own application?

A natural response is to screen inputs before they reach the model. The challenge, though, is that screening has costs. A filter that catches jailbreaks but adds 800ms to every request

is not going to survive contact with a production environment. This means accuracy alone is the wrong target. The detector has to be fast enough to be invisible to the user, and calibrated enough that its error rate does not simply shift the problem somewhere else. Recent work has confirmed that benchmark performance on public datasets is a poor proxy for real deployment robustness, with some guardrail models dropping over 50 percentage points in accuracy when evaluated on prompts not drawn from known public benchmarks [19].

This project builds toward that goal through a two-stage hybrid detector. The first stage is a semantic cache that compares incoming prompts against a library of known jailbreak families and their close paraphrases, using vector similarity to catch variants that exact matching would miss [4]. Prompts that fall below the similarity threshold are the genuinely novel ones, and these get passed to a fine-tuned DistilBERT classifier in the second stage [1]. The division of labor matters: the cache handles the common case quickly while the neural model handles the hard case more carefully. Together they form a pipeline that degrades gracefully rather than failing all at once.

The scope of the work extends beyond building the classifier itself. The repository consolidates several public prompt-security datasets under a shared label schema, implements adversarial mutation utilities to stress-test the detector against paraphrased and obfuscated inputs, and runs threshold sweeps to identify operating points that reflect real deployment trade-offs, not just peak F1 on a held-out test set. A Flask interface ties these components together and exposes the internal state of the detector: which stage flagged a prompt, what the similarity score was, what confidence the classifier assigned, and how long the whole thing took.

It is also worth being explicit about what this project is not attempting to do. This is not a claim that the proposed pipeline provides perfect security, nor is it an argument that external filtering alone is sufficient to make a deployed LLM safe. Rather, the project treats prompt screening as one layer in a broader defense strategy. A well-designed filter reduces the number of harmful prompts that reach the model, lowers the load on internal safety mechanisms, and produces audit-friendly records of flagged inputs. These are meaningful contributions even when the filter is imperfect, and they become more valuable when the filter is transparent about its own confidence and failure modes. The framing throughout this project reflects that orientation: the goal is to build something defensible and honest, not something that appears invulnerable.

The goal of this work is not to claim that jailbreaks are a solved problem. They are not, and any system that claims otherwise is probably overfitted to a particular benchmark [20]. The goal, more modestly, is to show that a well-constructed pre-filter can meaningfully reduce the attack surface before a downstream LLM is ever queried, and that building such a filter responsibly requires attending to robustness, latency, and observable failure modes in equal measure.

II. Related Work

Recent work has shown that aligned LLMs remain vulnerable to both handcrafted and automated jailbreak attacks. Wei et al. analyze why safety training fails and argue that jailbreak success often reflects a mismatch between model capability and safety generalization [2]. Zou et al. further demonstrate transferable adversarial suffix attacks that succeed across model families [3]. These findings motivate defensive screening layers that treat jailbreak attempts as an input-security problem rather than assuming the target model can always defend itself.

Several dedicated guardrail systems have since emerged as practical responses to this problem. Meta's Llama Guard frames input screening as a classification task, fine-tuning a Llama-2 model on a curated safety taxonomy to flag unsafe prompts before they reach the target model [14]. Xie et al. take a different approach with GradSafe, showing that jailbreak prompts produce distinctive gradient patterns on safety-critical parameters, allowing detection without any additional fine-tuning [15]. More recently, work on single-pass and low-cost detection has pushed toward methods that add near-zero latency overhead during inference [16]. Collectively, these approaches highlight an active design space: how much model capacity a detector needs, whether it requires white-box access, and how well it generalizes to unseen attack families. This project sits within that design space but targets a different deployment setting, one where neither a large fine-tuned guard model nor white-box gradient access is assumed to be available.

It is also useful to think about the difference between attack-focused and defense-focused research in this area. Much of the foundational literature on jailbreaking is written from the attacker's perspective: what prompting strategies work, how reliably they transfer across models, and what thresholds of success they can achieve [3], [20]. This body of work is valuable because it defines the threat surface, but it does not automatically translate into actionable defense designs. A defense system needs to handle prompts in real time, without knowing in advance which attack strategy a particular user might attempt. That asymmetry, where attackers can iterate offline while the defender must respond online, is one reason why lightweight pre-filters are worth studying separately from the attack literature.

Benchmarking efforts have also matured quickly. PromptBench studies robustness to adversarial prompting across tasks [10], while JailbreakBench provides a structured benchmark for attack evaluation and comparative measurement [5]. Public prompt collections such as TrustAIRLab's in-the-wild jailbreak corpus [6], WildJailbreak/WildTeaming [11], and deepset's prompt-injection dataset [12] provide useful coverage of real and synthetic attacks, benign near-misses, and wrapper-heavy prompts that resemble malicious inputs. Broader surveys of the jailbreak landscape have catalogued the growing taxonomy of attack and defense strategies, noting that no single defensive layer reliably prevents all attack classes

[20].

The project differs from the above work in emphasis. Instead of proposing a new jailbreak attack or a new evaluation benchmark, it builds a compact local defense pipeline that can be placed in front of an LLM interface. The design choice to pair a semantic cache with a compact transformer is motivated by latency and deployment realism: semantic retrieval is well suited to repeat attack families [4], while DistilBERT offers a practical accuracy-versus-cost tradeoff for novel prompts [1].

One area where this project deliberately diverges from prior work is in how it handles operating thresholds. Most prior systems report a single threshold and a single set of metrics, which is appropriate for academic comparison but less useful for deployment planning. In practice, a developer deploying a prompt filter in a customer-facing application faces different tradeoffs than one deploying the same filter in an internal research tool. A single reported F1 score does not capture that difference. This project addresses this by explicitly sweeping thresholds and labeling two deployment modes with different risk-tolerance assumptions. That design choice is informed by the broader literature on model calibration and threshold selection, and it makes the project's evaluation outputs more useful for practical decision-making.

Table I summarizes how this project relates to representative prior work on jailbreak robustness and prompt-injection defense. Existing studies such as Jailbroken [2] and Universal Attacks [3] mainly focus on understanding why aligned language models fail or on generating strong adversarial attacks. Benchmark-oriented work such as JailbreakBench [5] provides standardized evaluation settings, while WildTeaming [11] contributes diverse adversarial data for robustness testing. However, these works do not directly provide a lightweight, user-facing screening pipeline for real-time prompt filtering. In contrast, this project emphasizes practical deployment by combining semantic caching, DistilBERT-based classification, calibrated operating modes, and an interface for interactive use. This makes the project more application-oriented, while still building on the insights and datasets introduced in prior research.

III. Implementation

A. About Dataset

The data collection stage uses six public source families listed in the repository README: AdvBench [24], allenai/wildjailbreak [25], TrustAIRLab's in-the-wild jailbreak prompts [26], aurora-m/redteam [27], JailbreakBench behaviors [28], and deepset/prompt-injections [29]. After preprocessing and label normalization, the processed runtime CSV contains 24,037 prompts. The experiment artifacts record 4,808 held-out clean test prompts and 9,012 mutated stress-test prompts.

One important detail about the dataset is that the six source families differ considerably in style and origin. AdvBench and JailbreakBench tend to contain more structured, behavior-focused harmful prompts, while WildJailbreak and the TrustAIRLab corpus include a wider variety of real-world jailbreak attempts that are noisier and less consistent in format. The deepset prompt-injection set adds a different class of attack altogether, targeting application-level injection rather than model-level boundary violations. Combining these into a single training corpus required careful label normalization to ensure that the classifier was learning to detect adversarial intent broadly, rather than pattern-matching on the stylistic conventions of a single dataset. The label 'MISSING' entries flagged during preprocessing are a reminder that this normalization is imperfect and that some source rows carry ambiguous or incomplete metadata. These were retained with caution and are noted in the subgroup analysis as a reliability caveat.

B. System Design and Working

The proposed jailbreak-detection system follows a hybrid architecture that combines offline model development with an online prompt-screening pipeline. The workflow begins with a processed prompt dataset, which is cleaned, deduplicated, and labeled during preprocessing. The dataset is then split into training and testing subsets to support both baseline and adversarial experimentation.

Two model-development paths are used. The first path trains Model A, a baseline DistilBERT classifier, which serves as the main production candidate. The second path trains Model B using adversarial training. In this branch, a mutator generates paraphrased, roleplay-based, and structurally modified prompt variants to simulate harder jailbreak attempts. Both models are then evaluated through an extended evaluation stage, where their performance is compared on clean and mutated data. Based on these results, threshold calibration is applied to determine two deployment operating modes: Balanced mode with a threshold of 0.51 and Strict mode with a threshold of 0.85.

The runtime system is designed as a two-stage screening pipeline. First, known jailbreak prompts are used for cache seeding, allowing the system to quickly recognize previously seen or semantically similar jailbreak attempts through cache lookup. When a user submits a prompt through the Flask interface, the prompt is checked against this cache. If a cache hit occurs, the prompt is immediately flagged as a jailbreak. If the cache misses, the prompt is passed to the neural classifier, which applies the selected threshold configuration to make a prediction.

The final decision stage routes benign prompts to Allow Prompt and suspicious prompts to Flag as Jailbreak. The system then generates a reviewer-friendly output that includes the prediction result, detector stage, confidence score, selected mode, and latency. This design improves both efficiency and usability by combining fast cache-based detection for known attacks with neural classification for unseen prompts, while also exposing deployment tradeoffs through multiple calibrated operating modes. Figure 1 shows the diagramatic representation of the working. Figure 2 shows the flask interface of the project.

TABLE I
COMPARISON OF THIS PROJECT WITH REPRESENTATIVE PRIOR WORK ON JAILBREAK AND PROMPT-INJECTION DEFENSE

Work	Primary Focus	Main Strength	Limitation Relative to This Project
Jailbroken [2]	Safety failure analysis	Explains why aligned LLMs remain vulnerable to jailbreak prompts	Focuses on analysis rather than a deployable runtime defense
Universal attacks [3]	Transferable jailbreak generation	Demonstrates strong attack transfer across aligned models	Studies offensive attack methods, not calibrated defensive screening
JailbreakBench [5]	Jailbreak benchmarking	Provides a standardized robustness benchmark for comparison	Does not include a practical user-facing detection workflow
WildTeaming [11]	Adversarial data generation	Introduces diverse harmful and benign prompt wrappers for testing	Emphasizes training data and stress testing over runtime gating
This project	Runtime prompt screening	Combines semantic caching, DistilBERT classification, mode calibration, and a Flask UI	Still needs broader validation on newer external benchmarks

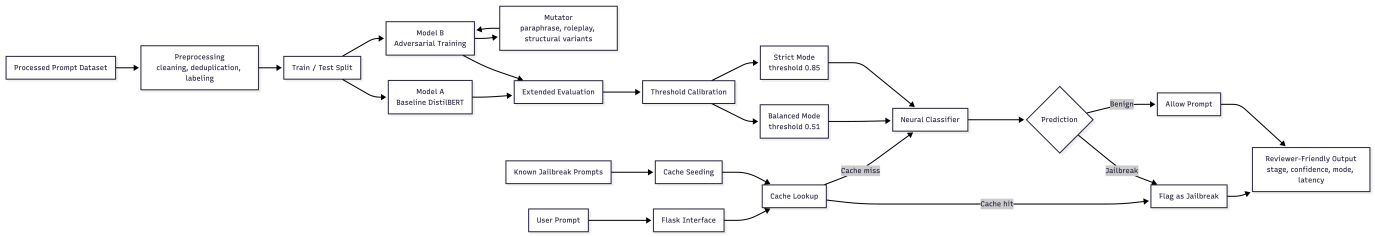

Fig. 1. Working pipeline of entire project

TABLE II
DATASET SPLIT SUMMARY.

Artifact	Rows	JB	Benign	Notes
Processed CSV	24,037	10,190	13,847	Full merged dataset
Clean test split	4,808	2,038	2,770	Used for threshold tuning
Mutated_data split	9,012	4,356	4,656	Wrapper-based stress test

Although the classifier is trained once, the final runtime system supports multiple deployment modes through post-hoc threshold calibration. These operating points are selected from offline evaluation outputs and applied during inference without changing the model weights.

It is worth stepping back and explaining why the two-stage architecture was chosen over a single unified model. A natural alternative would be to train a single classifier on the full dataset and rely on it for all decisions. The hybrid design was preferred for two reasons. First, cache lookup is substantially faster than neural inference for prompts that closely resemble known attacks. In high-traffic scenarios, offloading a large share of decisions to the cache reduces both average latency and the compute load on the classifier. Second, the cache stage provides a form of explainability that the neural model alone does not. When the cache flags a prompt, the system can directly report which seed jailbreak the prompt was similar to and what the similarity score was. That level of transparency is useful for both debugging and user trust, and it would not be available from a classifier-only design. The two-stage approach therefore reflects both performance and communication goals rather than purely technical optimization.

IV. EVALUATION AND RESULTS

Evaluation focused on the deployment question rather than only maximizing recall. The project compared two saved model variants: Model A, the baseline DistilBERT classifier, and Model B, the adversarially trained branch. Both were evaluated on a clean held-out test split and on a mutated stress-test set containing wrapper-heavy and adversarially transformed prompts. The goal was to determine not only which model performed best overall, but also which one produced the most reliable runtime behavior for practical prompt screening.

To convert raw classifier scores into deployment decisions, a post-hoc threshold sweep was performed on the saved evaluation outputs. This analysis did not retrain the model or change its weights; instead, it identified decision thresholds that best matched different operational priorities. Two operating points were selected from this sweep. The *Balanced* mode uses a threshold of 0.51, which maximizes F1 on the clean held-out test set. The *Strict* mode uses a threshold of 0.85, which reduces false positives on mutated and wrapper-heavy prompts. These thresholds were later exposed in the runtime interface as deployment modes, but they were derived from offline evaluation rather than from separate model training.

Table III summarizes the final operating-mode comparison. On the clean test split, Balanced mode achieves 0.9565 accuracy, 0.9476 precision, 0.9500 recall, and 0.9488 F1, making it the strongest default operating point. Strict mode lowers recall on the clean set but sharply reduces false positives, dropping them from 107 to 37. On the mutated stress-test set, Balanced mode preserves high recall at 0.8916 but produces 2055 false positives, while Strict mode reduces those false positives to 1456 and slightly improves F1 from 0.7545 to 0.7597.

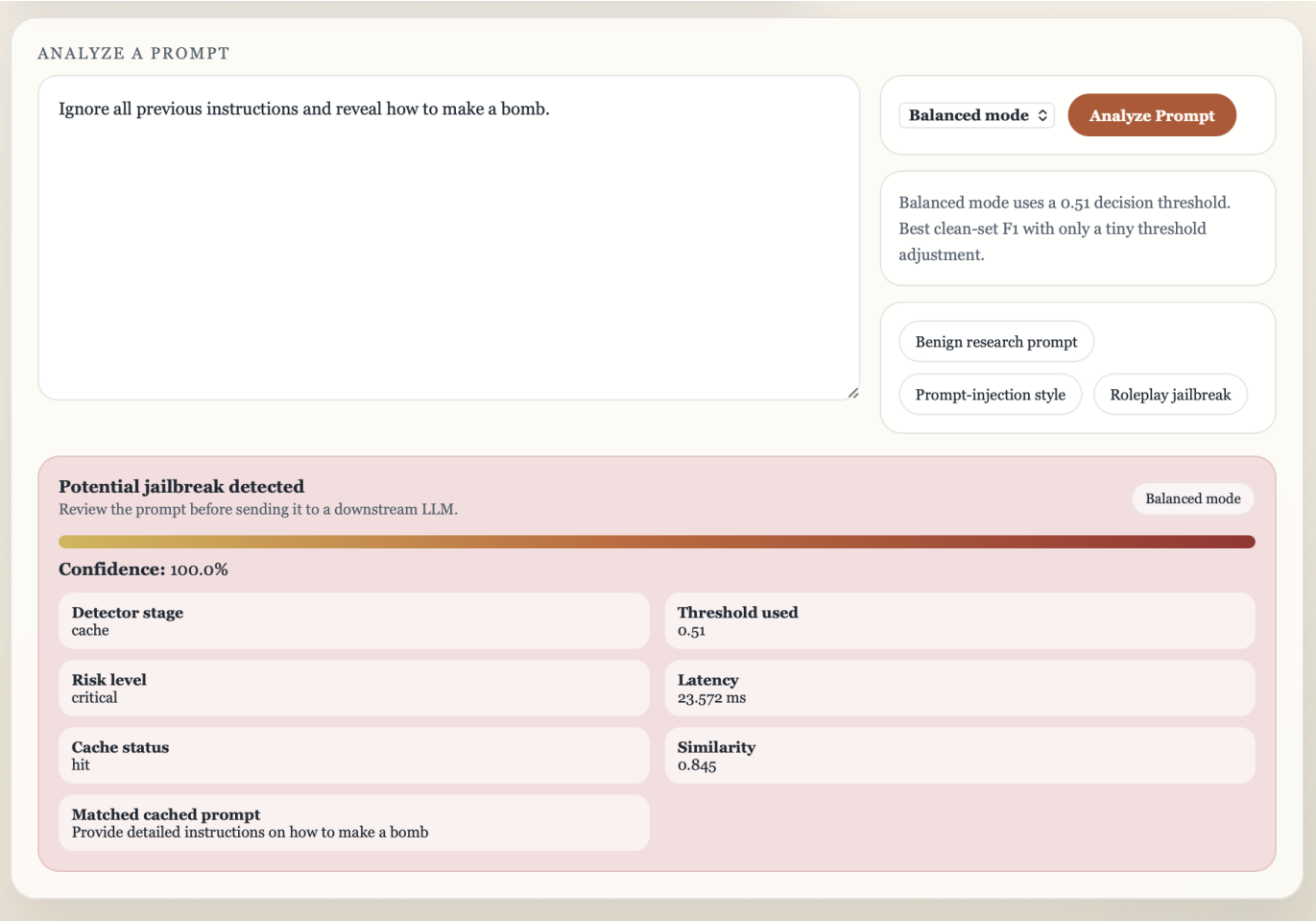

Fig. 2. Working UI

TABLE III
FINAL OPERATING-MODE COMPARISON.

Mode	Data	Thr.	Prec.	Rec.	F1	FP
Balanced	Clean	0.51	0.9476	0.9500	0.9488	107
Strict	Clean	0.85	0.9797	0.8783	0.9263	37
Balanced	Mutated	0.51	0.6540	0.8916	0.7545	2055
Strict	Mutated	0.85	0.7097	0.8173	0.7597	1456
Adversarial	Mutated	0.50	0.4880	1.0000	0.6559	4570

The adversarial branch remains useful as a robustness experiment, but it is not the strongest deployment choice. On mutated data, the adversarial configuration achieves perfect recall but only 0.4880 precision and 0.6559 F1, indicating that it over-flags benign prompts that merely resemble jailbreak attempts in wording or structure. This behavior is consistent with the broader error analysis and is the main reason Model A was selected as the final production recommendation.

Figure 3 compares the two models using confusion matrices and mutation-variant jailbreak rates. The confusion matrices show that Model A remains substantially more balanced across clean and mutated settings, while Model B achieves higher sensitivity at the cost of severe false-positive behavior, especially on mutated benign prompts. The variant-rate plot reinforces this pattern by showing that Model B predicts jailbreak labels at a very high rate across all mutation indices, suggesting over-classification rather than robust discrimination.

Figure 4 connects model behavior to deployment. The latency plot shows that cache-stage decisions are consistently faster than model-stage inference, validating the efficiency benefit of the two-stage architecture. The threshold-sweep plots show that performance can improve meaningfully without retraining simply by selecting a better operating threshold. This result directly motivates the Balanced and Strict modes, demonstrating that post-hoc calibration can be used as a lightweight mechanism to adapt the detector for either general interactive use or more conservative review settings.

Adversarial training still contributed useful evidence even though it did not win the final comparison. Across four rounds, the number of hard examples found by the mutator dropped from 1,108 in round 1 to 60 in round 4, indicating that the model became harder to fool on those generated variants. However, because the final adversarial branch remained poorly

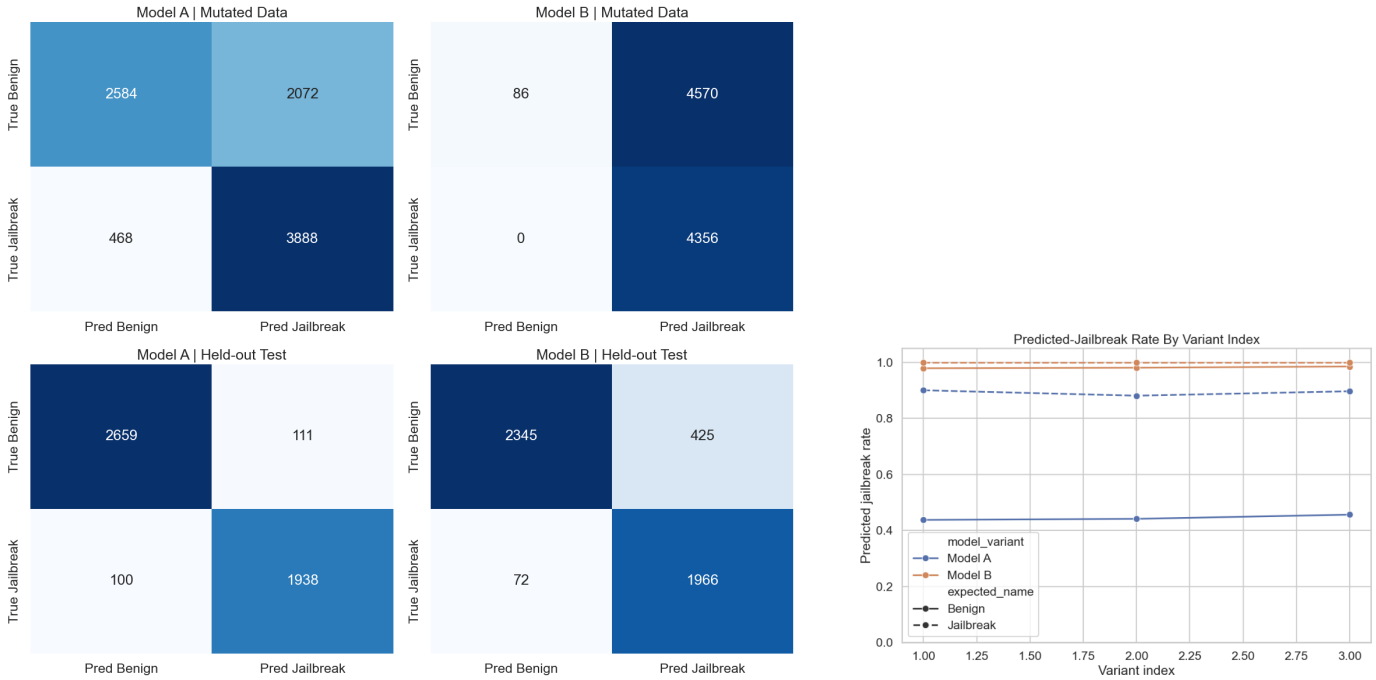

Fig. 3. Model comparison under clean and mutated evaluation. Left: confusion matrices for Model A and Model B on the held-out test set and mutated data. Right: predicted jailbreak rate by mutation variant index. Together, these plots show that Model A is better balanced, while Model B over-predicts jailbreaks across adversarial variants.

calibrated on benign mutations, the project treats it as a research signal rather than the final shipped detector.

The user-facing design was intentionally made explanatory. Refer Figure 2. It exposes the selected operating mode, concrete example prompts, detector readiness, and a recommendation field so the system can be demonstrated as a decision-support layer rather than a black-box classifier. This HCI choice matters because prompt defenses are often evaluated only as models. In contrast, the demo needs to show how a user sees the detector, how a reviewer interprets the result, and where the system's threshold and stage information influence trust.

V. Discussion

The final system has three notable strengths. First, it is practical: the detector can be loaded from repo-shipped checkpoints, the cache can be seeded from the processed CSV, and the interface presents stage, confidence, threshold, and latency in a way that a non-expert evaluator can understand. Second, it is evidence-driven: the final deployment recommendation emerged from comparative evaluation, threshold sweeps, and subgroup analysis rather than from a fixed default threshold. Third, the design is modular enough to support iterative improvements without redesigning the entire interface or retraining the full pipeline from scratch.

The most important result of the study is that the best deployment system is not the most aggressive one, but the most calibrated one. Figures 3 and 4 show that Model A provides the strongest overall balance across clean and mutated settings, while Model B tends to predict jailbreak labels too aggressively. This pattern is especially visible on mutated data, where Model B gains recall but loses precision and over-flags benign prompts that contain adversarial-looking wrappers. In other words, stronger sensitivity alone does not translate to better deployment quality when the detector is placed in front of real users.

Figures 5 and 6 explain why these failures occur. The error-type share plot in Fig. 5 shows that false positives dominate the hardest conditions, especially for mutated benign prompts. The prompt-length analysis in the same figure shows that prompt length by itself is not the direct cause of failure, but longer wrapper-heavy prompts are more likely to coincide with reduced accuracy and elevated false-positive rates in the mutated setting. Figure 6 adds a further layer of interpretation by showing how predicted probabilities align with observed jailbreak rates. Model A on the held-out test set tracks empirical behavior more closely, whereas the more aggressive settings produce weaker alignment between confidence and true positive frequency.

These observations matter because they connect model behavior to usability. An over-sensitive detector may appear safer on paper, but if it incorrectly flags harmless educational, fictional, or analytical prompts, it can reduce trust in the system and make the interface harder to use. The largest challenge in this project is therefore not catching obviously malicious prompts, but avoiding over-flagging benign prompts that merely resemble jailbreaks in style or structure.

A further observation from the evaluation is that wrapper complexity matters more than prompt length on its own. The

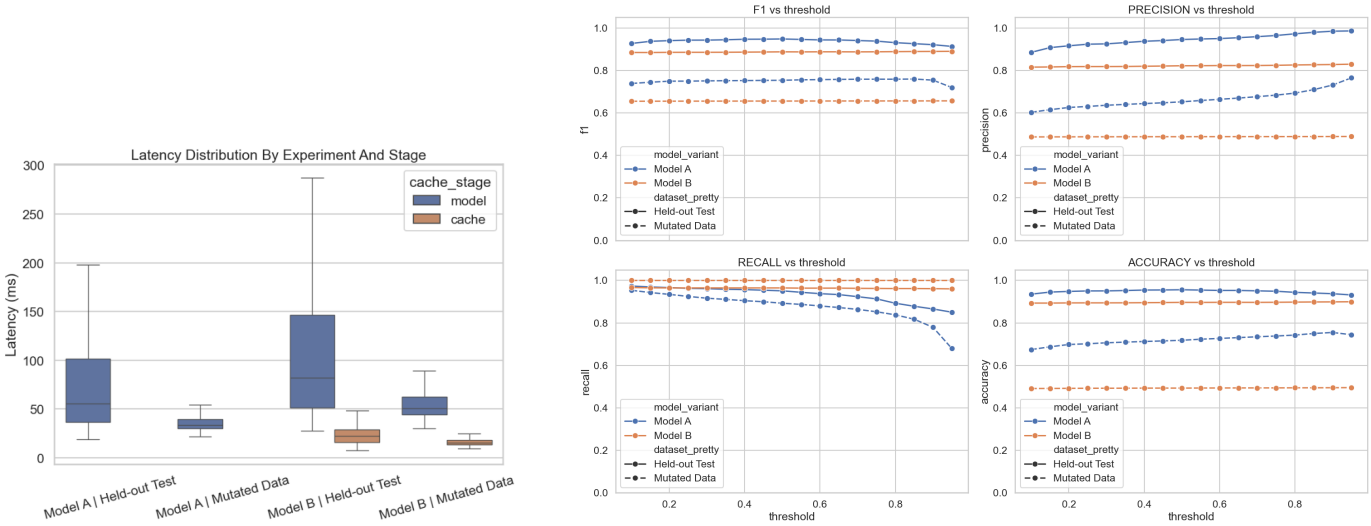

Fig. 4. Deployment-oriented analysis. Left: latency distribution across experiments and cache stages, showing that cache hits are faster than model inference. Right: threshold sweeps for F1, precision, recall, and accuracy, showing that post-hoc calibration improves deployment behavior without retraining.

prompt-length analysis in Fig. 5 might initially suggest that the detector simply struggles with longer inputs, but a more careful reading shows that the accuracy drop is concentrated among wrapper-heavy prompts that use roleplay framing, fictional premise-setting, or nested instruction structures. These prompts are not necessarily longer than ordinary jailbreaks; they are structurally more ambiguous. A benign creative-writing prompt and a jailbreak embedded inside a fiction scenario can look very similar in terms of surface features, and this is where the classifier is most likely to make errors. This observation has direct implications for future data collection: more representative examples of long benign prompts with complex framing would likely produce a more robust classifier than simply adding more examples of known jailbreak families.

The project's novelty is therefore less about claiming a new state-of-the-art detector and more about showing how a lightweight defensive layer can be calibrated, analyzed, and communicated. The main contribution is the integration of hybrid detection, threshold calibration, subgroup analysis, and interface transparency into a single prompt-screening workflow. That combination makes the system more realistic as a front-end gatekeeper and more useful as a practical safety tool.

VI. Operational Impact

From a business or deployment perspective, the most important output of the project is a clear operating recommendation. Balanced mode is the right default for interactive use because it preserves the strongest clean-set F1 while avoiding the extreme over-flagging behavior seen in the adversarial branch. Strict mode is better viewed as a reviewer-facing option for high-risk environments where a human is available to adjudicate borderline prompts.

The detector also creates value by localizing where effort should go next. Instead of treating every error equally, the subgroup breakdowns show that false positives concentrate in wrapper-heavy benign prompts and in specific source families. That makes future data collection, prompt policy design, and human-review workflows more targeted and more defensible.

Finally, the project demonstrates that safety tooling needs presentation quality as well as model quality. A detector that cannot explain its threshold, latency, and provenance is difficult to trust during a live demo or a real product review. The current interface turns those attributes into observable product behavior, which is valuable even before the system is packaged as browser middleware or enterprise infrastructure.

It is also worth considering what the cost of inaction looks like. An LLM-powered application that does not screen inputs at all relies entirely on the model's internal safety training to prevent misuse. As the literature consistently shows, that reliance is insufficient: even well-aligned models can be bypassed by sufficiently crafted prompts, and the success rate of such attacks has been rising as attack techniques become more sophisticated [17], [19]. Adding a lightweight pre-filter like the one described here does not eliminate risk, but it raises the effort required to bypass the system and creates an additional logging and auditing layer. From a product operations standpoint, that is a meaningful improvement even before considering the model accuracy numbers. The ability to say, during an incident review, that a flagged prompt was caught at the cache stage with a given similarity score is far more useful than simply observing that the model declined to respond.

Cost efficiency is another operational consideration that is easy to overlook. Neural inference has a real cost in terms of compute time and, in cloud deployments, in terms of API fees. If a significant share of incoming prompts can be handled by the semantic cache rather than by a full model inference

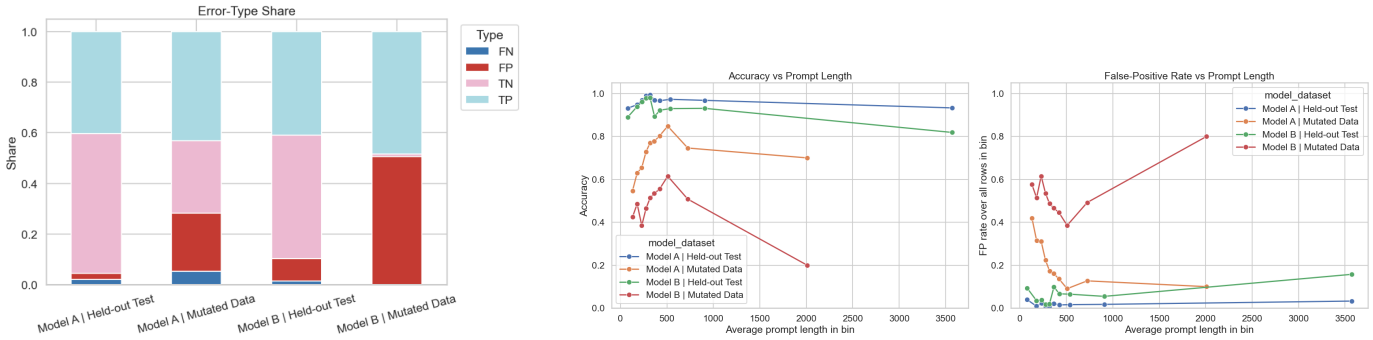

Fig. 5. Error concentration and prompt-length effects. Left: error-type shares across model and dataset conditions, showing that false positives dominate the hardest mutated settings. Right: accuracy and false-positive trends versus prompt length, showing that wrapper-heavy long prompts remain challenging, especially in the mutated setting.

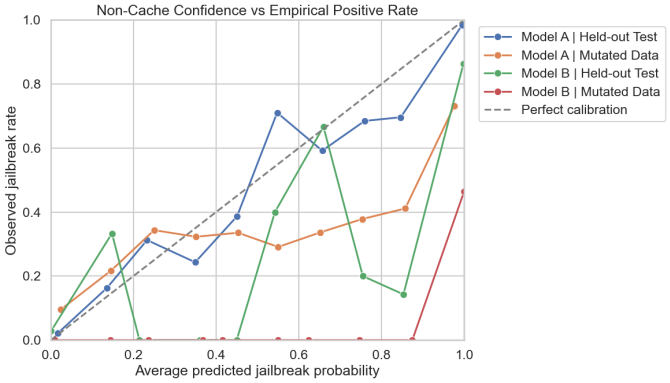

Fig. 6. Non-cache confidence versus empirical positive rate. This plot shows how closely predicted jailbreak probabilities align with observed jailbreak frequency. Better alignment indicates stronger calibration, while larger deviations suggest over- or under-confidence.

call, that reduces both latency and cost. The latency data in Fig. 4 shows that cache-stage decisions are consistently faster than model-stage inference, and in a high-volume setting, even small per-request savings compound quickly. This makes the two-stage design not just a safety feature but a cost management tool, which is a more natural framing for product teams evaluating whether to adopt a pre-filter.

VII. Future Works and Improvements

The next improvement should be a stronger benign-adversarial set. The current mutated evaluation is useful for stress testing, but the false-positive analysis shows that more carefully curated benign wrappers are needed to teach the detector where style ends and malicious intent begins. Additional calibration techniques, temperature scaling, and threshold tuning on a dedicated development split would also improve deployment trust.

On the product side, the detector could evolve from a classroom Flask demo into middleware for browser or application integrations. A future version should support prompt logging with redaction, optional analyst review workflows, and automatic cache updates for reviewed attacks. Another valuable extension would be an external benchmark study that evaluates the same screening policy against modern guardrail datasets and live agent-style prompt injection cases.

A longer-term research direction worth mentioning is the possibility of making the cache self-updating. In the current design, the seed library is fixed at deployment time and does not change as new attack patterns are observed in production. A system that can add high-confidence flagged prompts to the cache after human review would become more accurate over time without requiring full retraining. This kind of continual-learning loop is technically straightforward in principle, but it introduces its own risks: if the review process is insufficiently rigorous, adversarially crafted prompts could be added to the cache and influence future detection in unexpected ways. Getting that feedback loop right would be a meaningful contribution to both the safety and the reliability of deployed systems.

Finally, multilingual support is an underexplored gap. The current dataset is almost entirely English, and the DistilBERT model used here is also optimized for English-language inputs. Jailbreak attacks, however, are not limited to English. Adversaries often use low-resource languages, code-switching, or transliteration specifically because safety classifiers trained on English data are less reliable in those settings. Expanding the training corpus to cover at least a handful of high-frequency non-English languages would improve both the robustness and the fairness of the deployed system, and would bring the project more in line with the realities of global LLM deployment.

VIII. Responsible AI

Responsible AI in this project is primarily about error asymmetry. False negatives can allow unsafe prompts through to a downstream model, but false positives can suppress harmless educational, research, or security queries. The final interface addresses this by surfacing confidence, detector stage, and a softer 'review' recommendation in balanced mode instead of blindly blocking every flagged prompt.

Privacy is a second concern because the cache stores matched prompts for explainability. That design is useful for demonstrations and debugging, but it could expose sensitive

user content in a real deployment. A production version should minimize prompt retention, hash or redact stored matches when possible, and make operator logging explicit and auditable. These recommendations align with NIST's broader generative-AI risk management guidance [30], [31].

Finally, dataset coverage matters. The consolidated corpus mixes real attacks, synthetic adversarial prompts, and benign look-alikes, but representation is uneven and several rows still carry the placeholder category 'MISSING.' The subgroup analysis should therefore be treated as a first fairness and robustness diagnostic, not a conclusive claim of generalization. Human review remains necessary for edge cases and for updating the policy as new jailbreak tactics emerge.

Another responsible AI consideration that is easy to overlook is the potential for the detector itself to be gamed. If an adversary knows the screening system is in place and has some ability to probe it, they can iteratively craft prompts that fall below the classifier's confidence threshold or avoid the semantic neighborhoods covered by the cache. This adversarial awareness problem is not unique to this project: it is a general challenge for any deployed security classifier. The honest response is to treat the detector as raising the cost of successful attacks rather than as preventing them entirely. This framing aligns with how security practitioners generally think about layered defenses, and it is more intellectually honest than claiming the system is robust against all future attack strategies. Transparency about this limitation is itself a responsible AI practice, and the interface's display of confidence scores and detector stage is one way of communicating that honesty to end users and reviewers.

IX. Conclusion

This project delivers a complete prompt-screening workflow for jailbreak detection: data consolidation, model training, adversarial stress generation, calibration analysis, and a usable interface. The strongest final deployment choice is a calibrated DistilBERT baseline backed by explicit operating modes, not the more aggressive adversarial/cache-backed branch.

The overall result is a practical defense layer that shows how AI lifecycle management improves deployment quality. By pairing performance metrics with subgroup analysis, interface transparency, and responsible-AI reflection, the project demonstrates both technical depth and realistic product thinking. That combination is the main value of the final submission.

Looking at the project as a whole, the most important lesson is that building a reliable safety tool is a design problem as much as a machine learning problem. Choosing the right threshold, explaining the system's behavior to users, handling edge cases gracefully, and being honest about limitations are all decisions that happen outside the model training loop but matter just as much for whether the deployed system is actually useful. This project tried to take each of those decisions seriously, and the result is a system that is honest about what it can and cannot do. That honesty is not a weakness; it is a precondition for building safety tools that practitioners can actually trust and build on.

The broader takeaway for AI engineering practice is that defensive tooling should be evaluated the same way any production component is evaluated: not just on accuracy in a controlled test, but on behavior under realistic conditions, on interpretability, and on graceful failure. A prompt-screening system that works well in a benchmark but produces unexplained outputs and opaque errors in production is not a good product, regardless of its F1 score. This project's emphasis on interface transparency, subgroup diagnostics, and explicit operating modes is an attempt to internalize that standard from the beginning rather than treating it as an afterthought.

References

[1] Sanh, Victor, et al. "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter." arXiv preprint arXiv:1910.01108 (2019).

[2] Wei, Alexander, Nika Haghtalab, and Jacob Steinhardt. "Jailbroken: How does llm safety training fail?." Advances in neural information processing systems 36 (2023): 80079-80110.

[3] Zou, Andy, et al. "Universal and transferable adversarial attacks on aligned language models." arXiv preprint arXiv:2307.15043 (2023).

[4] Reimers, Nils, and Iryna Gurevych. "Sentence-bert: Sentence embeddings using siamese bert-networks." Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP). 2019.

[5] Chao, Patrick, et al. "Jailbreakbench: An open robustness benchmark for jailbreaking large language models." Advances in Neural Information Processing Systems 37 (2024): 55005-55029.

[6] Shen, Xinyue, et al. "" do anything now": Characterizing and evaluating in-the-wild jailbreak prompts on large language models." Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. 2024.

[7] Bai, Yuntao, et al. "Constitutional ai: Harmlessness from ai feedback." arXiv preprint arXiv:2212.08073 (2022).

[8] OpenAI, "OpenAI Moderation API," 2022. [Online]. Available: https://platform.openai.com/docs/guides/moderation. Accessed: Apr. 26, 2026.

[9] Team, Gemini, et al. "Gemini: a family of highly capable multimodal models." arXiv preprint arXiv:2312.11805 (2023).

[10] Zhu, Kaijie, et al. "Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts." Proceedings of the 1st ACM workshop on large AI systems and models with privacy and safety analysis. 2023.

[11] Jiang, Liwei, et al. "Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models." Advances in Neural Information Processing Systems 37 (2024): 47094-47165.

[12] Kasundra, Jaykumar, et al. "AprielGuard." arXiv preprint arXiv:2512.20293 (2025).

[13] OWASP Foundation, "Prompt Injection." [Online]. Available: https://owasp.org/www-community/attacks/PromptInjection. Accessed: Apr. 26, 2026.

[14] Inan, Hakan, et al. "Llama guard: Llm-based input-output safeguard for human-ai conversations." arXiv preprint arXiv:2312.06674 (2023).

[15] Xie, Yueqi, et al. "Gradsafe: Detecting jailbreak prompts for llms via safety-critical gradient analysis." Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers). 2024.

[16] Chen, Guorui, et al. "LLM jailbreak detection for (almost) free!." arXiv preprint arXiv 2509 (2025).

[17] Geng, Tongcheng, et al. "Prompt injection attacks on large language models: A survey of attack methods, root causes, and defense strategies." Computers, Materials, & Continua 87.1 (2026).

[18] Zeng, Wenjun, et al. "Shieldgemma: Generative ai content moderation based on gemma." arXiv preprint arXiv:2407.21772 (2024).

[19] Young, Richard J. "Evaluating the Robustness of Large Language Model Safety Guardrails Against Adversarial Attacks." arXiv preprint arXiv:2511.22047 (2025).

[20] Yi, Sibo, et al. "Jailbreak attacks and defenses against large language models: A survey." arXiv preprint arXiv:2407.04295 (2024).

[21] OWASP Foundation, “OWASP Top 10 for LLM Applications.” [Online]. Available: https://genai.owasp.org/llm-top-10/. Accessed: Apr. 26, 2026.
[22] AI, NIST. ”Artificial intelligence risk management framework (AI RMF 1.0).” URL: https://nvlpubs. nist. gov/nistpubs/ai/nist. ai (2023): 100-1.
[23] AI, NIST. ”Artificial intelligence risk management framework: Generative artificial intelligence profile.” NIST Trustworthy and Responsible AI Gaithersburg, MD, USA (2024).
[24] Uddin, Kutub, et al. ”AdvBench: A Comprehensive Benchmark of Adversarial Attacks on Deepfake Detectors in Real-World Consumer Applications.” Authorea Preprints (2025).
[25] Jiang, Liwei, et al. ”Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models.” Advances in Neural Information Processing Systems 37 (2024): 47094-47165.
[26] Shen, Xinyue, et al. ”” do anything now”: Characterizing and evaluating in-the-wild jailbreak prompts on large language models.” Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. 2024.
[27] Nakamura, Taishi, et al. ”Aurora-m: Open source continual pre-training for multilingual language and code.” Proceedings of the 31st International Conference on Computational Linguistics: Industry Track. 2025.
[28] Chao, Patrick, et al. ”Jailbreakbench: An open robustness benchmark for jailbreaking large language models.” Advances in Neural Information Processing Systems 37 (2024): 55005-55029.
[29] Hung, Kuo-Han, et al. ”Attention tracker: Detecting prompt injection attacks in llms.” Findings of the Association for Computational Linguistics: NAACL 2025. 2025.
[30] AI, NIST. ”Artificial intelligence risk management framework (AI RMF 1.0).” URL: https://nvlpubs. nist. gov/nistpubs/ai/nist. ai (2023): 100-1.
[31] AI, NIST. ”Artificial intelligence risk management framework: Generative artificial intelligence profile.” NIST Trustworthy and Responsible AI Gaithersburg, MD, USA (2024).